03. Variables & Data Types

Created by Li Huanyu, last modified by suncaper on Mar 05, 2026

Introduction

In programming, you constantly need to store, manipulate, and retrieve data. To do this, we use **Variables** and **Data Types**. Think of a variable as a labeled box on a warehouse shelf. The **Data Type** defines what *kind* of item is stored inside that box (e.g., text, whole numbers, or decimals).

In this lesson, we will cover:

- **Variables** (How to create and name those boxes).
- **Dynamic Typing** (How Python differs from legacy Big Data languages).
- **The Big Three Data Types:** Strings, Integers, and Floats.

Part 1: Variables (The Data Containers)

Unlike other programming languages, Python has no special command (like `var` or `let`) for declaring a variable. A variable is created the moment you first assign a value to it using the equals sign (=).

1. Correct Example ✅

```
user_age = 25       # Python automatically knows this is a number
user_name = "John"  # Python automatically knows this is text
```

2. Incorrect Example ❌ Variable names must follow strict rules: they **cannot** start with a number, and they **cannot** contain spaces.

```
1st_user = "John"   # <--- SyntaxError: invalid decimal literal
user name = "John"  # <--- SyntaxError: invalid syntax (no spaces allowed)
```

Best Practice: In modern Data Engineering (and Python in general), we use **snake_case** for variable names. All lowercase, separated by underscores (e.g., `raw_data_path`, `max_retries`).

Python vs. The "Statically Typed" Languages

In languages like Java or Scala (the native languages of Hadoop and Apache Spark), you must explicitly tell the computer the *Data Type* before you create the variable. This is called **Static Typing**. Python uses **Dynamic Typing**, meaning Python acts like a smart assistant and figures it out for you on the fly.

1) Java / Scala Example (The "Strict" Way)

```
// Java or Scala Syntax
int userAge = 25;           // You MUST declare it as an 'int' (Integer)
String userName = "John";   // You MUST declare it as a 'String'
```

2) Python Example (The "Clean" Way)

```
# Python Syntax
user_age = 25          # Python sees 25 and says, "Ah, that's an integer!"
user_name = "John"     # Python sees quotes and says, "Ah, that's a string!"
```

3) Side-by-Side Comparison

Feature	Java / Scala (Legacy Big Data)	Python (Modern Data Engineering)
Declaring Text	`String city = "NY";`	`city = "NY"`
Declaring Numbers	`int count = 5;`	`count = 5`
Type Flexibility	Hard (Types are locked)	Easy (A variable can hold an `int`, then later hold a `str`)

> **The "Big Data" Context:**
>
> Why does this matter? Python makes writing scripts incredibly fast. However, Big Data databases (like SQL, Snowflake) and processing engines (like PySpark DataFrames) **strictly care** about data types. If your Python variable holds text, but the database expects a number, your pipeline will crash. That's why we must deeply understand the data types we are working with.

Part 2: The Core Data Types & `type()`

Data types matter because they determine what operations you can perform. You can multiply numbers, but you can't logically multiply two words.

1. Strings (`str`) Strings are sequences of characters (text). You can use either single quotes (`'`) or double quotes (`"`), but you must be consistent.

```
first_name = "Alice"
last_name = 'Smith'
database_query = "SELECT * FROM users"
```

2. Integers (`int`) Integers are whole numbers, positive or negative, without decimals. They are primarily used for counting, ID numbers, or loop iterations.

```
total_rows = 1000000
retry_attempts = 3
temperature = -5
```

3. Floats (`float`) Floats are "floating-point numbers"—numbers that contain a decimal point. They are used when precision is needed (money, percentages, weights).

```
item_price = 19.99
sensor_reading = 3.14159
discount_rate = 0.05
```

How to check what's in the box: `type()`

When reading data from a file, you might not know what type it is. Use Python's built-in `type()` function to inspect it.

```
mystery_data = 42.0
print(type(mystery_data))  # Output: <class 'float'>
```

Part 3: Hands-on Exercise (Pipeline Configuration)

In Data Engineering, you often write configuration files to set up your pipeline parameters before the actual data processing begins. Let's write one.

1. Open PyCharm.
2. Create a new file named `variables_practice.py`.
3. Type the following code:

```
# ---------------------------------------------------------
# Big Data Simulation: Pipeline Configuration Variables
# ---------------------------------------------------------

"""
This script simulates setting up variables for a nightly
data processing pipeline.
"""

# Defining Variables (Python infers the data types dynamically)
pipeline_name = "Nightly_Sales_ETL"   # String
batch_size = 50000                    # Integer (Rows per batch)
failure_threshold = 0.05              # Float (5% error rate allowed)

# Let's inspect our setup
print("--- Pipeline Config ---")
print("Job Name:", pipeline_name, "| Type:", type(pipeline_name))
print("Batch Size:", batch_size, "| Type:", type(batch_size))
print("Error Threshold:", failure_threshold, "| Type:", type(failure_threshold))

print("\n--- Testing Data Type Logic ---")
# Math works on ints and floats
next_batch = batch_size + 10000
print("Next batch size will be:", next_batch)
```

```
# ❌ What happens if we try to do math with a String?
# Uncomment the line below to see a TypeError in action!
# print(pipeline_name + 10)

Run the code. Output:

--- Pipeline Config ---
Job Name: Nightly_Sales_ETL | Type: <class 'str'>
Batch Size: 50000 | Type: <class 'int'>
Error Threshold: 0.05 | Type: <class 'float'>

--- Testing Data Type Logic ---
Next batch size will be: 60000
```

Summary

- **Variables** are created instantly with `=`. Use `snake_case` for names.
- **Dynamic Typing:** Python automatically detects data types (unlike Java/Scala).
- **Strings (`str`)** hold text and require quotes.
- **Integers (`int`)** hold whole numbers.
- **Floats (`float`)** hold decimal numbers.
- Use `type()` to verify the data type of any variable.

No labels